

What Writing Assistants Can Learn from Programming IDEs

Sergey Titov
JetBrains Research
Paphos, Republic of Cyprus
sergey.titov@jetbrains.com

Agnia Sergeyuk
JetBrains Research
Belgrade, Republic of Serbia
agnia.sergeyuk@jetbrains.com

Timofey Bryksin
JetBrains Research
Limassol, Republic of Cyprus
timofey.bryksin@jetbrains.com

ABSTRACT

With the development of artificial intelligence, writing assistants (WAs) are changing the way people interact with text, creating lengthy outputs that can be overwhelming for users. The programming field has long addressed this issue, and Integrated Development Environments (IDEs) have been created for efficient software development, helping programmers reduce the cognitive load. This experience could be employed in the development of WAs. IDEs can also be used to test assumptions about interventions that help people interact with WAs efficiently. Previous works have successfully used self-written IDE plugins to test hypotheses in the field of human-computer interaction. The lessons learned can be applied to the building of WAs.

1 INTRODUCTION

Writing assistants (WAs) are changing the way people interact with text. These tools can be incredibly useful in improving the quality and efficiency of writing [7]. However, with recent progress in the development of large language models such as GPT-3 [3] and MT-NLG [17], WAs can create long output [16] that can sometimes be overwhelming and difficult to manage for anyone and especially for neurodiverse users. Creating tools that aim to reduce cognitive load during writing-related tasks can help address this problem.

The programming field has long been working on this issue. Code writing is a mentally challenging task that imposes much cognitive load on the user. For this reason, Integrated Development Environments (IDEs), an ecosystem of WAs, have been created to make code writing more efficient and less demanding for mental effort [6, 8, 10]. Over the last several decades, a lot of work has already been done to support users' performance by incorporating programming language peculiarities into IDEs (e.g., keyword coloring, code writing conventions, and fixed context). However, there is still a long way to go. To embrace inclusivity, thorough research should be carried out on possible interventions to reduce the cognitive load of various groups of users. As an ever-changing industry-level tool, an IDE is an excellent instrument for that: tasks, changes, and measurements can be easily incorporated into the system by plugins to test hypotheses [5, 13].

We argue that established practices in the development of IDEs, as well as the use of IDEs to research WAs, can be fruitful for next-generation WAs development.

2 IDE EXPERIENCE FOR WRITING ASSISTANTS

2.1 IDE's ways to reduce cognitive load

IDEs are advantageous for efficient software development. To provide assistance, IDEs leverage the hallmarks of programming languages and development practices. The usefulness of this could

be assessed, since IDEs allow us to collect various metrics that, in turn, reflect users' efficiency and other characteristics of their performance [1, 2].

In this section, we discuss some techniques that are used in IDEs to help programmers reduce the cognitive load and suggest how these techniques could be used in the next generation of WAs in general writing domains.

Code coloring. Unlike other texts, code has a structure-based readability improvement: the coloring of keywords and literals (e.g., strings or numbers). IDEs take advantage of this by using color coding with respect to the programming language's syntax. That makes it easier for programmers to quickly recognize the structure of the code snippet and follow its logic. Although structural word coloring is mostly unique to programming languages, there is some design space where we can use it in AI-based WAs. For example, various machine learning explainability tools use coloring to help the user understand and interpret the results of the model [12, 14]. Similarly, in WAs, colors can be used to highlight key parts of a paragraph that are most important to convey a specific message or idea. Such a feature might be especially useful in academic and law writing, where the coherence of argumentation is critical. In those domains, word coloring would make it easier for a writer to organize their thoughts.

Code style conventions. Coding style is a formally defined set of conventions that helps to support consistency in a codebase and makes it easier to comprehend. As a result, these conventions can affect both the performance of users, as well as the maintainability of the code. IDEs use code style information to automatically format the code appropriately, making it easier for programmers to write code that is consistent and adheres to best practices. There are some existing conventions in the world of writing — every domain has traditions and rules for composing and delivering text, e.g., citation formatting rules, or scientific paper structure. The field could greatly benefit from WAs that are capable of converting plain language text into any required form or convention. Integrating conventions into WAs or even inferring and learning conventions from corpora of texts in a particular organization can help maintain the consistency and readability of texts, especially in the domains of formal and business communication.

Explicit context. In software development, code is typically placed within a file surrounded by other code it might interact with, the project has a set of used libraries and other parts of the configured environment, and usually there is a history of edits as part of a version control system. All of this is available to a programmer in their IDE. IDEs emphasize this context and suggest code snippets that are relevant to it (for example, suggesting code completions based on the imported libraries), making it easier for programmers to write code quickly that integrates seamlessly with the rest of the project. Bringing context to WAs might help better understand

the structure of the text and keep track of changes made to the documents. Using and emphasizing terminology as a context can assist writers in supporting their degree of accuracy and clarity in academic, legal, and technical writing. In turn, version control would be a valuable feature for WAs in any writing domain where collaboration and frequent revisions are expected. While some of this functionality, like version control and document structure visualization, has already been implemented in tools such as Overleaf and Google Docs, other useful applications of these approaches are yet to be developed.

Applying ideas from IDEs to WAs presents unique challenges that must be conquered. For example, context analysis in IDEs involves algorithms like traversing dependency graphs or analyzing the project structure. In the context of the more ambiguous setting of a general text, it is necessary first to develop a structure-agnostic system using natural language processing techniques to find a relevant context in an array of user files. Additionally, design challenges for implementing new features in WAs are emerging. For instance, how to provide comprehensible multiline suggestions. While evolving and augmenting a wide range of features and functionality, writing tools would need to have a user-friendly, adaptable interface that allows writers to quickly access and use features without disrupting users' workflow. The tool should not be too distracting or intrusive because that could hinder the writing process and compromise the author's creativity. As a whole, the proposed technology transfer is both rewarding and challenging.

2.2 IDE as a playground for WA research

Some modern programming languages such as Python and Swift are designed to be as close to a human language as possible, while also having strict rules and syntax that ensure that the computer can execute the code. Thus, the code could be perceived as an instance of text [9]. Therefore, IDE might be used as a model of WAs to test assumptions about interventions to help people interact with tools efficiently.

A wide variety of data collected and stored in an IDE makes it possible to evaluate and optimize the features of the environment. By measuring, for instance, the readability of the code before and after using some IDE feature, researchers and practitioners can assess the effectiveness of the provided interventions and draw corresponding conclusions. In previous work, this approach has already been tested by using a custom IDE plugin to test hypotheses from the field of human-computer interaction [4, 11, 15, 18, 19]. The same could be done for topics related to WAs. Although there are certain differences between the mediums, some lessons learned from the development of the IDEs can be useful for the building of the WAs in general and conversely, lessons from WAs can make IDEs more useful.

3 CONCLUSION

With the development of artificial intelligence, new concerns in the field of WAs are emerging. One of them is that the amount of information processed by the tool and by its users becomes increasingly large and overwhelming, especially for people with

neurodiversity. Here we propose that this issue can be addressed by using the decades-long experience of the development of IDE and by using them as a model for future research.

REFERENCES

- [1] Anant P Bhardwaj, Dave Luciano, and Scott R Klemmer. 2011. Redprint: integrating API specific" instant example" and" instant documentation" display interface in IDEs. In *Proceedings of the 24th annual ACM symposium adjunct on User interface software and technology*. 21–22.
- [2] Jacob T Biehl, Mary Czerwinski, Greg Smith, and George G Robertson. 2007. FASTDash: a visual dashboard for fostering awareness in software teams. In *Proceedings of the SIGCHI conference on Human factors in computing systems*. 1313–1322.
- [3] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [4] Daniel Kyle Davis and Feng Zhu. 2022. Analysis of software developers' coding behavior: A survey of visualization analysis techniques using eye trackers. *Computers in Human Behavior Reports* 7 (2022), 100213.
- [5] Nikolai Denisov. 2021. Creating an educational plugin to support online programming learning A case of IntelliJ IDEA plugin for A+ Learning Management System. (2021).
- [6] Sarah Fakhoury, Devjeet Roy, Yuzhan Ma, Venera Arnaudova, and Olusola Adesope. 2020. Measuring the impact of lexical and structural inconsistencies on developers' cognitive load during bug localization. *Empirical Software Engineering* 25 (2020), 2140–2178.
- [7] Katy Gero, Alex Calderwood, Charlotte Li, and Lydia Chilton. 2022. A design space for writing support tools using a cognitive process model of writing. In *Proceedings of the First Workshop on Intelligent and Interactive Writing Assistants (In2Writing 2022)*. 11–24.
- [8] Ted Habbeck, Larry Koved, and Marco Pistoia. 2008. Security WORKbench Development Environment 4 Java. In *Symposium on Usable Privacy and Security (SOUPS)*.
- [9] Abram Hindle, Earl T Barr, Mark Gabel, Zhendong Su, and Premkumar Devanbu. 2016. On the naturalness of software. *Commun. ACM* 59, 5 (2016), 122–131.
- [10] Haley Hunter-Zinck, Alexandre Fioravante De Siqueira, Valeri N Vázquez, Richard Barnes, and Ciera C Martinez. 2021. Ten simple rules on writing clean and reliable open-source scientific software. . e1009481 pages.
- [11] Vseslav Kasatskii, Agnia Serheyuk, Anastasiia Serova, Sergey Titov, and Timofey Bryksin. 2023. The Effect of Perceptual Load on Performance within IDE in People with ADHD Symptoms. *arXiv preprint arXiv:2302.06376* (2023).
- [12] Scott M Lundberg and Su-In Lee. 2017. A Unified Approach to Interpreting Model Predictions. In *Advances in Neural Information Processing Systems* 30, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.). Curran Associates, Inc., 4765–4774. <http://papers.nips.cc/paper/7062-a-unified-approach-to-interpreting-model-predictions.pdf>
- [13] João Ramos, Paulo Cunha, and Fábio Silva. 2022. A Tool to Assess Students' Performance in Programming. In *Methodologies and Intelligent Systems for Technology Enhanced Learning, 11th International Conference 11*. Springer, 166–174.
- [14] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. 2016. "Why should i trust you?" Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*. 1135–1144.
- [15] André L Santos and Brad A Myers. 2017. Design annotations to improve API discoverability. *Journal of Systems and Software* 126 (2017), 17–33.
- [16] Shuming Shi, Enbo Zhao, Duyu Tang, Yan Wang, Piji Li, Wei Bi, Haiyun Jiang, Guoping Huang, Leyang Cui, Xinting Huang, et al. 2022. Effidit: Your AI Writing Assistant. *arXiv preprint arXiv:2208.01815* (2022).
- [17] Shaden Smith, Mostofa Patwary, Brandon Norick, Patrick LeGresley, Samyam Rajbhandari, Jared Casper, Zhun Liu, Shrimai Prabhumoye, George Zerveas, Vijay Korthikanti, et al. 2022. Using deepspeed and megatron to train megatron-turing nlq 530b, a large-scale generative language model. *arXiv preprint arXiv:2201.11990* (2022).
- [18] Frank F Xu, Bogdan Vasilescu, and Graham Neubig. 2022. In-ide code generation from natural language: Promise and challenges. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 31, 2 (2022), 1–47.
- [19] Vlas Zyrianov, Drew T Guarnera, Cole S Peterson, Bonita Sharif, and Jonathan I Maletic. 2020. Automated recording and semantics-aware replaying of high-speed eye tracking and interaction data to support cognitive studies of software engineering tasks. In *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 464–475.